UNIVERSIDADE DO VALE DO RIO DOS SINOS (UNISINOS)

CURSO DE ENGENHARIA DA COMPUTAÇÃO

GUIA DE DOMÍNIO TÉCNICO E DEFESA DE BANCA

MÓDULO 4: Classificador XGBoost (Extreme Gradient Boosting)

Função Objetivo Regularizada $\longrightarrow$ Expansão de Taylor de 2ª Ordem $\longrightarrow$ Score de Similaridade $\longrightarrow$ Cálculo do Gain

---

Candidato: Andrio Gabriel Epping | **Papel:** Orientador Técnico & Examinador Rigoroso de Banca |
Data: Novembro de 2026

Apresentação do Módulo de Treinamento

Este guia é dedicado ao domínio matemático, algorítmico e computacional do **Módulo 4: Classificador XGBoost (Extreme Gradient Boosting)**, o motor de indução e tomada de decisão diagnóstica utilizado em ambos os eixos (Track A e Track B) do Trabalho de Conclusão de Curso (TCC).

O foco deste documento é capacitar o estudante a deduzir analiticamente todas as etapas teóricas do XGBoost: a expansão de Taylor de segunda ordem da função de perda, a dedução do peso ótimo das folhas (w_j^*), o cálculo do Score de Similaridade, a fórmula formal do Ganho de Partição ($Gain$) com custo de complexidade γ, o tratamento multiclasse via Softmax Cross-Entropy com cálculo exato do vetor gradiente (g_i) e hessiana (h_i), o manuseio de desbalanceamento por `sample_weight` e `CustomSMOTE`, bem como os hiperparâmetros ótimos identificados pelo IGA e as respostas ideais para a sabatina da banca examinadora.

Sumário

1 Pilar 1: Fundamentação Matemática e Teórica **3**
- 1.1 O Modelo Aditivo de Boosting e Representação de Árvores . . . 3
- 1.2 Função Objetivo Global Regularizada . . . 3
- 1.3 Aproximação de Segunda Ordem por Série de Taylor . . . 3
- 1.4 Dedução Analítica do Peso Ótimo Foliar e Score de Similaridade . . . 4
- 1.5 Cálculo Exato do Ganho de Partição (Split Finding Gain) . . . 5
- 1.6 Formulação Multiclasse com Softmax Cross-Entropy . . . 5
- 1.7 Ponderação por `sample_weight` . . . 6

2 Pilar 2: Rastreamento no Código (Linha a Linha) **6**
- 2.1 Treinamento do XGBoost no Track A (`src/iga_xgboost/train_hybrid.py`) . . . 6
- 2.2 Rastreamento no Track B (`src/iga_xgboost/train_classification.py`) . . . 6
- 2.3 Tabela de Shapes e Fluxo de Tensores no XGBoost . . . 7

3 Pilar 3: Decisões de Projeto e Nuances do Meu Trabalho **7**
- 3.1 Por que XGBoost e não Random Forest, LightGBM ou Redes Neurais? . . . 7
- 3.2 Hiperparâmetros Finais Encontrados pelo IGA e o Efeito de Regularização . . . 8
- 3.3 Estratégias de Tratamento do Desbalanceamento: `sample_weight` vs. SMOTE . . . 8

4 Pilar 4: Sabatina da Banca (3 Perguntas Difíceis e Gabaritos) 8

1 Pilar 1: Fundamentação Matemática e Teórica

1.1 O Modelo Aditivo de Boosting e Representação de Árvores

O algoritmo XGBoost (*Extreme Gradient Boosting*) baseia-se no princípio do aprendizado em conjunto aditivo (*Additive Ensemble Learning*). Dado um conjunto de treinamento $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$, com n amostras, $\mathbf{x}_i \in \mathbb{R}^p$ atributos e rótulos $y_i \in \{0, \ldots, C-1\}$, a previsão do modelo após a inclusão de M árvores de decisão CART (*Classification and Regression Trees*) é expressa por:

$$\hat{y}_i^{(M)} = \sum_{m=1}^{M} f_m(\mathbf{x}_i), \quad f_m \in \mathcal{F} \tag{1}$$

onde $\mathcal{F} = \{f(\mathbf{x}) = w_{q(\mathbf{x})}\}$ representa o espaço funcional de árvores de decisão.

Cada árvore $f_m(\mathbf{x})$ é caracterizada por duas estruturas formais:

1. **Função de Mapeamento Estrutural (q):** $q : \mathbb{R}^p \to \{1, \ldots, T_L\}$, que direciona a instância $\mathbf{x}$ até o índice de uma folha terminal específica;
2. **Vetor de Pesos das Folhas (w):** $\mathbf{w} = [w_1, w_2, \ldots, w_{T_L}]^\top \in \mathbb{R}^{T_L}$, onde w_j é a pontuação numérica contínua (*logit*) atribuída a todas as amostras que caem na folha j.

1.2 Função Objetivo Global Regularizada

Em cada iteração $t \in \{1, \ldots, M\}$ do processo de *gradient boosting*, uma nova árvore $f_t(\mathbf{x})$ é inserida com o objetivo de minimizar a perda residual acumulada somada a termos explícitos de penalização da complexidade estrutural da árvore recém-criada:

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} l\left(y_i,\ \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)\right) + \Omega(f_t) \tag{2}$$

onde $l(y, \hat{y})$ é a função de perda convexa e diferenciável.

O termo de regularização $\Omega(f_t)$, introduzido por Chen & Guestrin (2016) para controlar o compromisso entre viés e variância, é formalizado por:

$$\Omega(f_t) = \gamma T_L + \frac{1}{2}\lambda \sum_{j=1}^{T_L} w_j^2 + \alpha \sum_{j=1}^{T_L} |w_j| \tag{3}$$

- γ (`gamma`): Custo de penalização por cada folha adicional criada na árvore, atuando diretamente como limiar de poda;
- λ (`reg_lambda`): Fator de regularização L_2 (Ridge) sobre as magnitudes dos pesos das folhas, impedindo pontuações extremas e reduzindo a variância;
- α (`reg_alpha`): Fator de regularização L_1 (Lasso) sobre os pesos das folhas, induzindo esparsidade ao zerar o peso de folhas com baixo suporte estatístico.

1.3 Aproximação de Segunda Ordem por Série de Taylor

Diferentemente do Gradient Boosting clássico de Friedman (que utiliza apenas aproximação de primeira ordem via descida do gradiente), o XGBoost emprega a expansão em Série de Taylor de segunda ordem da função de perda em torno da previsão do passo anterior $\hat{y}_i^{(t-1)}$:

$$l\left(y_i,\ \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)\right) \approx l\left(y_i, \hat{y}_i^{(t-1)}\right) + g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t^2(\mathbf{x}_i) \tag{4}$$

onde g_i (Gradiente) e h_i (Hessiana) são definidos como:

$$g_i = \left[\frac{\partial l(y_i, \hat{y})}{\partial \hat{y}}\right]_{\hat{y}=\hat{y}_i^{(t-1)}}, \qquad h_i = \left[\frac{\partial^2 l(y_i, \hat{y})}{\partial \hat{y}^2}\right]_{\hat{y}=\hat{y}_i^{(t-1)}} \tag{5}$$

Como o termo $l(y_i, \hat{y}_i^{(t-1)})$ é uma constante na iteração t (não depende de f_t), ele pode ser removido da otimização, resultando na **função objetivo simplificada de segunda ordem**:

$$\tilde{\mathcal{L}}^{(t)} = \sum_{i=1}^{n}\left[g_i f_t(\mathbf{x}_i) + \frac{1}{2}h_i f_t^2(\mathbf{x}_i)\right] + \gamma T_L + \frac{1}{2}\lambda\sum_{j=1}^{T_L} w_j^2 + \alpha\sum_{j=1}^{T_L}|w_j| \tag{6}$$

1.4 Dedução Analítica do Peso Ótimo Foliar e Score de Similaridade

Definindo $I_j = \{i \in \{1,\ldots,n\} \mid q(\mathbf{x}_i) = j\}$ como o conjunto de instâncias mapeadas para a folha terminal j, pode-se agrupar a soma sobre instâncias i em uma soma sobre as folhas $j \in \{1,\ldots,T_L\}$, já que $f_t(\mathbf{x}_i) = w_j, \forall i \in I_j$:

$$\tilde{\mathcal{L}}^{(t)} = \sum_{j=1}^{T_L}\left[\left(\sum_{i\in I_j} g_i\right) w_j + \frac{1}{2}\left(\sum_{i\in I_j} h_i + \lambda\right) w_j^2 + \alpha|w_j|\right] + \gamma T_L \tag{7}$$

Definindo as quantidades agregadas do nó folha j:

$$G_j = \sum_{i\in I_j} g_i, \qquad H_j = \sum_{i\in I_j} h_i \tag{8}$$

Dedução do Peso Ótimo Foliar (w_j^*)

Considerando a regularização L_2 pura ($\alpha = 0$), a função objetivo para a folha j é uma parábola estritamente convexa em relação a w_j:

$$J(w_j) = G_j w_j + \frac{1}{2}(H_j + \lambda)w_j^2 \tag{9}$$

Derivando em relação a w_j e igualando a zero:

$$\frac{\partial J(w_j)}{\partial w_j} = G_j + (H_j + \lambda)w_j = 0 \implies \mathbf{w}_j^* = -\frac{G_j}{H_j + \lambda} \tag{10}$$

Quando a regularização Lasso L_1 está ativa ($\alpha > 0$), a solução é obtida pelo operador de encolhimento suave (*Soft-Thresholding*):

$$\mathbf{w}_j^* = -\frac{\text{sign}(G_j)\cdot\max(0,\ |G_j| - \alpha)}{H_j + \lambda} \tag{11}$$

Substituindo o peso ótimo w_j^* de volta na Equação (6) (para $\alpha = 0$), obtém-se o valor ótimo da função objetivo para uma dada estrutura de árvore q:

$$\tilde{\mathcal{L}}^*(q) = -\frac{1}{2}\sum_{j=1}^{T_L}\frac{G_j^2}{H_j + \lambda} + \gamma T_L \tag{12}$$

O termo $\dfrac{G_j^2}{H_j + \lambda}$ é formalmente denominado **Score de Similaridade** (*Similarity Score*) da folha j. Ele quantifica a redução do erro residual ponderada pela curvatura (hessiana) das amostras que residem naquela folha.

1.5 Cálculo Exato do Ganho de Partição (Split Finding Gain)

Na construção das árvores, o algoritmo avalia a divisão de um nó pai I em dois nós filhos: I_L (esquerdo) e I_R (direito), onde $I = I_L \cup I_R$. Pela conservação dos gradientes:

$$G = G_L + G_R, \qquad H = H_L + H_R \tag{13}$$

Equação do Ganho de Divisão de Nós (*Gain*)

O ganho obtido pela partição é a diferença entre a impureza do nó pai e a soma das impurezas dos filhos, descontado o custo de criar uma nova folha (γ):

$$\text{Gain} = \frac{1}{2}\left[\underbrace{\frac{G_L^2}{H_L+\lambda}}_{\text{Similaridade Filho Esq.}} + \underbrace{\frac{G_R^2}{H_R+\lambda}}_{\text{Similaridade Filho Dir.}} - \underbrace{\frac{(G_L+G_R)^2}{H_L+H_R+\lambda}}_{\text{Similaridade Nó Pai}}\right] - \gamma \tag{14}$$

Critério de Poda Estrutural Automática:
Se Gain ≤ 0, a divisão não é realizada (ou é podada recursivamente no processo de pós-poda). O parâmetro γ funciona exatamente como um limiar de corte de significância estatística de segunda ordem.

1.6 Formulação Multiclasse com Softmax Cross-Entropy

Nos Tracks A (4 classes) e B (7 classes), o problema de diagnóstico é multiclasse. O XGBoost utiliza a função de perda Multi-Logloss (`eval_metric="mlogloss"`). Para cada amostra i, o modelo produz um vetor de pontuações brutas $\hat{\mathbf{y}}_i = [\hat{y}_{i,0}, \ldots, \hat{y}_{i,C-1}]^\top$.

As probabilidades a posteriori são obtidas pela função Softmax:

$$p_{i,c} = P(y_i = c \mid \mathbf{x}_i) = \frac{\exp(\hat{y}_{i,c})}{\sum_{k=0}^{C-1}\exp(\hat{y}_{i,k})}, \quad c \in \{0, \ldots, C-1\} \tag{15}$$

A função de perda Cross-Entropy de uma amostra i com rótulo real codificado em *one-hot* $\mathbf{y}_i = [y_{i,0}, \ldots, y_{i,C-1}]^\top$ é:

$$l(\mathbf{y}_i, \hat{\mathbf{y}}_i) = -\sum_{c=0}^{C-1} y_{i,c}\ln(p_{i,c}) \tag{16}$$

Gradiente e Hessiana na Classificação Multiclasse

O vetor gradiente de primeira ordem para a classe c é dado pela diferença entre a probabilidade estimada e o rótulo real:

$$g_{i,c} = \frac{\partial l}{\partial \hat{y}_{i,c}} = p_{i,c} - y_{i,c} \in [-1,\ 1] \tag{17}$$

A derivada de segunda ordem (Hessiana) expressa a curvatura da função de perda:

$$h_{i,c} = \frac{\partial^2 l}{\partial \hat{y}_{i,c}^2} = p_{i,c}(1 - p_{i,c}) \in \left[0,\ \frac{1}{4}\right] \tag{18}$$

Propriedade Crítica da Hessiana:
Quando o modelo possui alta incerteza sobre a classe ($p_{i,c} \approx 0{,}50$), a hessiana atinge seu valor máximo ($h_{i,c} = 0{,}25$), conferindo grande estabilidade ao denominador $H_j + \lambda$. Quando o modelo já tem certeza quase absoluta ($p_{i,c} \to 1$ ou $p_{i,c} \to 0$), a hessiana aproxima-se de zero ($h_{i,c} \to 0$), demonstrando que essas amostras não contribuem com nova curvatura para a divisão.

1.7 Ponderação por `sample_weight`

Quando uma amostra i possui um peso diferenciado $w_i > 0$ atribuído para balanceamento de classes:

$$g_{i,c}^{\text{pond}} = w_i \cdot g_{i,c}, \qquad h_{i,c}^{\text{pond}} = w_i \cdot h_{i,c} \tag{19}$$

As instâncias de classes minoritárias (como PD ou T1) recebem pesos multiplicativos mais altos, elevando sua contribuição tanto no numerador G_j quanto no denominador H_j, forçando o algoritmo a criar ramificações específicas para capturá-las.

2 Pilar 2: Rastreamento no Código (Linha a Linha)

2.1 Treinamento do XGBoost no Track A (`src/iga_xgboost/train_hybrid.py`)

O treinamento do modelo no Track A com os 4 gases e classes Katser:

```python
def _treinar_xgboost(
    X_train: np.ndarray,
    y_train: np.ndarray,
    hparams: dict,
    seed: int,
    sample_weight: np.ndarray | None = None,
) -> XGBClassifier:
    """Treina um XGBClassifier com os parametros decodificados pelo IGA."""
    modelo = XGBClassifier(
        **hparams,
        tree_method="hist",          # Algoritmo de quantizacao em histogramas
        eval_metric="mlogloss",      # Perda multiclasse cross-entropy
        random_state=seed,           # Reprodutibilidade estrita
        n_jobs=-1,                   # Paralelizacao total de CPU
        verbosity=0,
    )
    modelo.fit(X_train, y_train, sample_weight=sample_weight)
    return modelo
```

2.2 Rastreamento no Track B (`src/iga_xgboost/train_classification.py`)

No Track B, o classificador gerencia 7 classes da norma IEC 60599:

```python
LABEL_MAP_B: dict[str, int] = {
    "Normal": 0, "PD": 1, "D1": 2, "D2": 3, "T1": 4, "T2": 5, "T3": 6,
}

def treinar_modelo_final(
    X_train: np.ndarray,
    y_train: np.ndarray,
    hparams: dict,
    features_selecionadas: list[str],
    seed: int = RANDOM_SEED,
) -> XGBClassifier:
    clf = XGBClassifier(
        **hparams,
        tree_method="hist",
        eval_metric="mlogloss",
        random_state=seed,
        n_jobs=-1,
        verbosity=0,
    )
    clf.fit(X_train, y_train)
    return clf
```

2.3 Tabela de Shapes e Fluxo de Tensores no XGBoost

A Tabela 1 detalha a passagem de dados na entrada e saída do classificador.

Tabela 1: Evolução Dimensional das Matrizes no Classificador XGBoost.

Variável / Matriz	**Origem no Código**	**Dimensões (Shape)**	**Descrição no Problema**
`X_train` (Track A C2)	`katser_features_train.csv`	$(1785, 17)$	1.785 transformadores $\times$ 17 atributos
`X_train` (Track A C1)	Filtrado por $\mathbf{m}^*_{\text{IGA}}$	$(1785, 8)$	1.785 transformadores $\times$ 8 atributos ótimos
`y_train` (Track A)	`LABEL_MAP_A`	$(1785,)$	Rótulos inteiros $\{0, 1, 2, 3\}$
`X_test` (Track A)	`katser_features_test.csv`	$(900, 17)$ ou $(900, 8)$	900 transformadores de teste independente
`X_train` (Track B C3)	`china_features_train.csv`	$(1372, 19)$ ou $(1372, 16)$	1.372 instâncias de treino brutas / filtradas
`X_train_smote` (Track B)	Saída do `CustomSMOTE`	$(N_{\text{smote}}, 19)$	Treino sobreamostrado para classes raras
`X_test` (Track B)	`china_features_test.csv`	$(295, 19)$ ou $(295, 16)$	295 amostras reais deduplicadas de teste
`predict_proba()`	Invocação do modelo treinado	(N_{teste}, C)	Matriz de probabilidades Softmax por classe
`predict()`	$\arg\max_c p_{i,c}$	$(N_{\text{teste}},)$	Diagnóstico pontual predito como inteiro

3 Pilar 3: Decisões de Projeto e Nuances do Meu Trabalho

3.1 Por que XGBoost e não Random Forest, LightGBM ou Redes Neurais?

Justificativa da Seleção do XGBoost

1. **Desempenho Comprovado em Dados Tabulares Heterogêneos:** Conforme demonstrado empiricamente por Grinsztajn et al. (2022), ensembles impulsionados por gradiente superam redes neurais profundas (MLP, ResNet tabular) em dados tabulares onde atributos possuem escalas físicas heterogêneas e distribuições fortemente assimétricas (como ppm de gases e razões estequiométricas).

2. **Cálculo Exato e Rápido de Shapley via TreeSHAP:** O algoritmo TreeSHAP (Lundberg et al., 2020) opera com complexidade polinomial $\mathcal{O}(MLd^2_{\max})$ sobre árvores de decisão. Para modelos de caixa-preta como redes neurais ou SVM com kernel RBF, o cálculo de SHAP depende do KernelSHAP, cuja amostragem de Monte Carlo é exponencial e instável.

3. **Algoritmo `tree_method="hist"`:** O particionamento baseado em histogramas discretiza

as variáveis contínuas em até 256 caixas (*bins*). Isso reduz a complexidade de busca de divisão de nós de $\mathcal{O}(n \cdot p \log n)$ para $\mathcal{O}(n_{\text{bins}} \cdot p)$, permitindo que o IGA execute dezenas de milhares de dobras de treinamento em tempo hábil.

3.2 Hiperparâmetros Finais Encontrados pelo IGA e o Efeito de Regularização

A Tabela 2 compara os hiperparâmetros de referência fixa com os ótimos descobertos pelo IGA no Track A e Track B.

Análise Crítica de Engenharia:
Um questionamento óbvio é por que o IGA selecionou profundidade $d_{\max} = 12$ no Track B se o dataset contém apenas 1.372 amostras de treino (sendo classes como T1 e T2 com menos de 80 amostras). A resposta teórica é que o modelo não sofreu sobreajuste porque o IGA **compensou a profundidade com forte regularização Ridge ($\lambda = 3{,}94$), penalização de nós ($\gamma = 1{,}44$) e subamostragem extrema de colunas ($s_{\text{col}} = 0{,}34$)**. Cada árvore só enxerga cerca de um terço dos atributos por divisão, atuando de forma similar a uma floresta aleatória regularizada com capacidade de modelar interações termodinâmicas profundas.

3.3 Estratégias de Tratamento do Desbalanceamento: `sample_weight` vs. SMOTE

Por que usar abordagens diferentes para lidar com o desbalanceamento?

- **No Track A (Uso de `sample_weight`):** A base Katser possui 2.100 séries no treino, com desbalanceamento moderado (a classe majoritária Normal possui $\approx 80\%$, mas as minoritárias PD, D1 e T1 possuem dezenas de amostras). O XGBoost gerencia essa proporção perfeitamente multiplicando os gradientes e hessianas pelo peso da classe ($w_c = N/(C \cdot N_c)$) via `sample_weight`, evitando gerar dados sintéticos em séries temporais dinâmicas.
- **No Track B (Uso de `CustomSMOTE`):** A base de teste deduplicada contém apenas 16 amostras de T1 e 16 amostras de T2. Com um suporte amostral tão diminuto, o `sample_weight` puro provocaria alta instabilidade (uma única amostra de T1 teria peso excessivo, dominando a Hessiana H_j e gerando árvores com alta variância). O `CustomSMOTE` sintetizou novos vetores diretamente no espaço dos 5 gases brutos e recalculou as 14 variáveis derivadas, densificando o suporte estocástico das classes raras.

4 Pilar 4: Sabatina da Banca (3 Perguntas Difíceis e Gabaritos)

Pergunta Crítica da Banca: 1. Dedução dos Gradientes e Hessianas da Perda Multi-Logloss

"Na regressão com perda quadrática (MSE), a Hessiana é uma constante trivial ($h_i = 1$). No entanto, o seu XGBoost foi configurado com a perda multiclasse Softmax (`eval_metric="mlogloss"`). Mostre matematicamente como se calculam o gradiente $g_{i,c}$ e a hessiana $h_{i,c}$ a partir da função de perda Cross-Entropy multiclasse. O que acontece com a Hessiana quando o classificador tem quase 100% de certeza sobre uma classe ($p_{i,c} \to 1$) ou quando considera a classe quase impossível ($p_{i,c} \to 0$)? Como isso afeta o cálculo do Ganho de Partição?"

Tabela 2: Comparação dos Hiperparâmetros do XGBoost: Default vs. Otimizados pelo IGA.

Hiperparâmetro	**Default (C2/C4)**	**Track A IGA (C1)**	**Track B IGA (C3)**	**Papel Teórico e Efeito Prático**
`learning_rate` (η)	0,10	0,128	0,0596	Passo de encolhimento; valor menor no Track B evita sobreajuste
`max_depth` ($d_{\max}$)	6	12	12	Profundidade máxima; permite capturar interações entre 3+ gases
`n_estimators` (M)	100	422	342	Número de árvores sequenciais na convergência do resíduo
`subsample` (s_{row})	0,90	1,00	0,965	Amostragem estocástica de linhas por árvore
`colsample_bytree` (s_{col})	1,00	0,945	**0,343**	Amostragem de apenas 34% das colunas no Track B (alta diversidade)
`gamma` (γ)	0,00	0,00	**1,436**	Custo de divisão; poda ramos

Gabarito Técnico Ideal de Resposta (Blindagem de Banca)

Estratégia de Resposta:
Apresente a dedução clássica da entropia cruzada com Softmax e evidencie a interpretação da curvatura na convergência:

1. **Dedução do Gradiente de Primeira Ordem ($g_{i,c}$):** "Seja a perda de entropia cruzada para uma instância i com vetor *one-hot* real $\mathbf{y}_i$:

$$l(\mathbf{y}_i, \hat{\mathbf{y}}_i) = -\sum_{k=0}^{C-1} y_{i,k} \ln(p_{i,k}), \quad p_{i,k} = \frac{e^{\hat{y}_{i,k}}}{\sum_j e^{\hat{y}_{i,j}}} \tag{20}$$

   Pela regra da cadeia da derivada do Softmax ($\frac{\partial p_{i,k}}{\partial \hat{y}_{i,c}} = p_{i,c}(\delta_{k,c} - p_{i,k})$), a derivada parcial em relação ao logit da classe c resulta em:

$$g_{i,c} = \frac{\partial l}{\partial \hat{y}_{i,c}} = -\sum_{k=0}^{C-1} y_{i,k} \frac{1}{p_{i,k}} \frac{\partial p_{i,k}}{\partial \hat{y}_{i,c}} = p_{i,c}\left(\sum_k y_{i,k}\right) - y_{i,c} = \mathbf{p_{i,c}} - \mathbf{y_{i,c}} \tag{21}$$

   O gradiente é exatamente o resíduo da probabilidade prevista em relação ao rótulo real."

2. **Dedução da Hessiana de Segunda Ordem ($h_{i,c}$):** "Derivando o gradiente $g_{i,c}$ novamente em relação a $\hat{y}_{i,c}$:

$$h_{i,c} = \frac{\partial g_{i,c}}{\partial \hat{y}_{i,c}} = \frac{\partial (p_{i,c} - y_{i,c})}{\partial \hat{y}_{i,c}} = \mathbf{p_{i,c}(1 - p_{i,c})} \tag{22}$$

3. **Comportamento nos Casos Limites e Impacto no Ganho:**
   - Quando o modelo já tem certeza absoluta ($p_{i,c} \to 1$ para a classe verdadeira ou $p_{i,c} \to 0$ para uma classe errônea), o produto $p_{i,c}(1 - p_{i,c}) \to 0$, fazendo com que a hessiana $h_{i,c}$ convirja para zero.
   - Na fórmula do Ganho (Gain), o denominador é $H + \lambda$. Se todas as amostras de uma folha já estiverem classificadas com alta confiança ($p \to 1$), $H_j \approx 0$ e $G_j \approx 0$, resultando em Gain ≤ 0 após a subtração de γ. Isso impede que o XGBoost gaste divisões desnecessárias refinando amostras já perfeitamente resolvidas, concentrando as novas árvores exclusivamente nos transformadores com diagnósticos incertos ou fronteiriços."

Pergunta Crítica da Banca: 2. Profundidade 12 com 1.372 Amostras de Treino (295 de Teste): Como Evitar Ov

"Na Tabela do TCC, o modelo IGA do Track B (C3) opera com profundidade máxima $d_{\max} = 12$. Uma árvore binária com profundidade 12 pode gerar teoricamente até $2^{12} = 4.096$ folhas terminais. Considerando que o conjunto de treino deduplicado contém 1.372 amostras (e o teste 295), essa árvore teria mais folhas potenciais do que o número total de amostras disponíveis! Na teoria estatística, isso é o cenário clássico de sobreajuste catastrófico (memorização de dados). Como o seu modelo obteve $F_1 = 0{,}7970$ no teste (0,8000 nas 6 classes de falha) e não memorizou o treino? Demonstre formalmente quais mecanismos matemáticos impediram esse colapso."

Gabarito Técnico Ideal de Resposta (Blindagem de Banca)

Estratégia de Resposta:
Explique a interação tripla entre os mecanismos de regularização do XGBoost (γ, λ, w_{child} e s_{col}) que limitam a expansão efetiva da árvore muito antes do teto teórico:

1. **A Profundidade $d_{\max}$ é Apenas um Limite Superior (Teto), Não uma Obrigação:** "O parâmetro `max_depth=12` define apenas a profundidade máxima permitida, mas a árvore não é construída como uma árvore binária completa de 4.096 folhas. Três restrições matemáticas impediram que ela atingisse essa dimensão:
   (a) **Penalização de Custo por Folha ($\gamma = 1{,}436$):** Uma nova partição só é aceita se o ganho de impureza superar γ (Gain $> 1{,}436$). Como demonstrado na Equação (14), divisões que separam apenas 1 ou 2 amostras ruidosas geram ganhos próximos de zero e são eliminadas pelo algoritmo na etapa de poda reversa (*pruning*).
   (b) **Peso Mínimo Foliar (`min_child_weight = 3`):** Uma folha só pode ser criada se a soma de suas hessianas for no mínimo 3 ($\sum_{i \in I_j} h_i \geq 3$). Como a hessiana máxima de uma amostra multiclasse é $h \leq 0{,}25$, uma folha exige o suporte equivalente acumulado de pelo menos 12 instâncias incertas, tornando impossível criar folhas com 1 ou 2 amostras isoladas.
   (c) **Subamostragem Drástica de Colunas ($s_{\text{col}} = 0{,}343$):** A cada nó, o XGBoost sorteia estocasticamente apenas 34% das variáveis disponíveis (cerca de 5 ou 6 atributos dos 16 selecionados). Isso descorrelaciona as árvores, simulando a injeção de diversidade de uma floresta aleatória e impedindo que atributos com picos de ruído dominem todas as ramificações.
2. **Regularização Ridge Amortecedora ($\lambda = 3{,}942$):** O peso ótimo da folha é $w_j^* = -G_j/(H_j + 3{,}942)$. A adição desse termo de quase 4 unidades no denominador amortece o *logit* de saída, garantindo que mesmo folhas profundas recebam pontuações pequenas que só alteram a probabilidade final de forma gradual e ponderada pela taxa de aprendizado suave ($\eta = 0{,}0596$)."

Pergunta Crítica da Banca: 3. A Diferença entre `sample_weight` e SMOTE na Otimização do XGBoost

"Por que no Track A você utilizou a ponderação por instâncias (`sample_weight`), enquanto no Track B você desenvolveu uma classe própria de sobreamostragem sintética (`CustomSMOTE`)? Qual é a diferença teórica de otimização na expansão de Taylor do XGBoost entre reponderar instâncias existentes e gerar pontos sintéticos no espaço dos gases?"

Gabarito Técnico Ideal de Resposta (Blindagem de Banca)

Estratégia de Resposta:
Demonstre domínio sobre como pesos afetam o cálculo de derivadas e justifique a escolha pela densidade geométrica das classes no espaço de dados:

1. **O Efeito do `sample_weight` na Otimização de Segunda Ordem:** "Ao atribuir um peso $w_i > 1$ a uma instância de classe minoritária, a função de perda sofre uma homotetia local: $g_i^{\text{pond}} = w_i \cdot g_i$ e $h_i^{\text{pond}} = w_i \cdot h_i$. No cálculo do peso da folha:

$$w_j^* = -\frac{\sum_i w_i g_i}{\sum_i w_i h_i + \lambda} \tag{23}$$

   O `sample_weight` aumenta o peso do erro daquela amostra nas decisões de divisão, mas

não altera a geometria do suporte amostral. Ele apenas diz ao algoritmo para não errar aquele ponto específico."

2. **A Fragilidade do `sample_weight` com Classes Extremamente Raras (Track B):** "No Track B, classes como T1 e T2 possuem apenas 16 amostras de teste. Se utilizássemos apenas `sample_weight`, o peso atribuído a cada ponto seria gigantesco. Se uma única dessas 16 amostras contiver um erro de sensor cromatográfico ou estiver na fronteira geométrica de Duval, ela exercerá uma força desproporcional sobre o gradiente G_j, distorcendo a árvore inteira para acomodar um ponto ruidoso (alta variância)."
3. **A Vantagem do `CustomSMOTE` no Track B:** "O `CustomSMOTE` interpola os $k = 5$ vizinhos mais próximos no espaço contínuo dos gases brutos, criando uma **densificação contínua da variedade riemanniana da classe** (manifold). Isso fornece ao XGBoost múltiplos exemplos interpolados, permitindo que as árvores aprendam uma fronteira de partição suave e estável em vez de criar um nó folha agudo em torno de um único ponto real com peso artificialmente inflado. Já no Track A, como a base possui 2.100 séries completas e suporte estatístico suficiente, o `sample_weight` foi suficiente e evitou gerar séries temporais sintéticas que poderiam violar a física de difusão contínua."

Fim do Módulo 4 – Domínio Pleno do Classificador XGBoost para a Banca!